AUDK32 CLI examples based on TeraTerm test script

HB Rev. 1.10

Introduction

This document describes CLI-based examples provided by AUDK32 (ABOV Unified Development Kit for 32bit) and introduces how to test on-chip peripherals using TeraTerm/Minicom script.

Reference document

The following documents can be downloaded at ABOV's website www.abovsemi.com:

- STD10-HB_AUDK32 Guide_Eng.pdf
- AUDK32 Release Note
- AUDK32 API Reference in Windows help (*.chm) format
- Starter Kit documents of ABOV 32bit MCUs

Contents

Introduction 1
Reference document 1
1. Introduction 5
1.1 ABOV boards 5
1.1.1 Bare board detail 6
1.1.2 Limitations 7
1.2 Script location 8
2. Environment identification 9
2.1 Main components for S/W development 9
2.2 Build flags 9
3. Examples on bare board 10
3.1 User interface 10
3.2 Default commands 11
3.2.1 ls 11
3.2.2 lm 11
3.2.3 cm 11
3.2.4 help 12
3.2.5 h 12
3.2.6 h opt 13
3.3 Peripheral test with terminal script 14
3.3.1 Test script 15
3.3.2 How to use test script 17
3.3.3 Checking the result detail of test script 18
3.3.4 TeraTerm TTL command 18
4. Test script detail 19
4.1 TIMER1 19
4.1.1 01_timer1_intr_period.ttl 19
Revision history 22

List of tables

Table 1. Definition of ABOV boards 6
Table 2. S/W development environment 9
Table 3. Keil build flags for S/W development 9
Table 4. Eclipse build flag for S/W development 9

List of figures

Figure 1. Bare board configuration based on user input 6
Figure 2. Script location 8
Figure 3. Running example on bare target 10
Figure 4. Result of ‘ls’ command 11
Figure 5. Result of ‘lm’ command 11
Figure 6. Result of ‘cm’ command 11
Figure 7. Result of ‘help’ command 12
Figure 8. Result of ‘h’ command 12
Figure 9. Result of ‘h opt’ command 13
Figure 10. AUDK32 example demonstration using TeraTerm script 14
Figure 11. TeraTerm script file of peripherals 15
Figure 12. Example of test script 16
Figure 13. Selection of macro file 17
Figure 14. Result of macro in TeraTerm 18
Figure 15. H/W configuration block diagram of 01_timer1_intr_period.ttl 19

1. Introduction

All examples provided in AUDK32 are implemented based on a standard HAL (Hardware Abstraction Layer) API. AUDK32 also provides a list of CLI commands to execute a test item or the scenario implemented by a target example for validating on-chip peripheral.

The CLI commands can be executed by two ways as the below:

- Users type the CLI commands on any serial host program directly.
  - The users have to consider about the composition of the commands provided by the target example for a test scenario.
  - The commands are described in detail in the following chapters.
- Users execute TeraTerm or Minicom script file depending on OS of host PC.

  Even if the macro syntax of TeraTerm is different from Minicom, they all must use the CLI commands provided by the target example and support the same test scenario.
  - The all-script files consist of a group of CLI commands on top of the macros defined by TeraTerm or Minicom to build up a test scenario.
  - Serial host program is supposed to read the script file up and does the actions described by the macros including sending CLI commands to an example running on the target board.

This effectively enables users to easily test a peripheral, reducing the time and cost involved in project development.

1.1 ABOV boards

Example given by AUDK32 is closely related to the target board, the H/W platform on which it operates.

Table 1 lists the boards that ABOV typically provides.

Table 1. Definition of ABOV boards

Board	Description
Bare	• Bare doesn't mean an actual board but virtual one. • The Most peripherals on ABOV 32bit devices are commonly supported. The example selected for a supported peripheral to test can be run or tested on any platform. In this case, we don't need a specific board but common board. This is why bare is common virtual platform.
Starter Kit	Starter Kit has LEDs and some buttons only.
STKS	• STKS stands for a Starter Kit Shield Board and has extended features that cannot be tested on Starter Kit alone. • As STKS does not have an MCU, it must be connected to Starter Kit via I/O pin connector. • STKS typically deploys functions such as IR and display (7-segment or LCD) controlled by Starter Kit.

1.1.1 Bare board detail

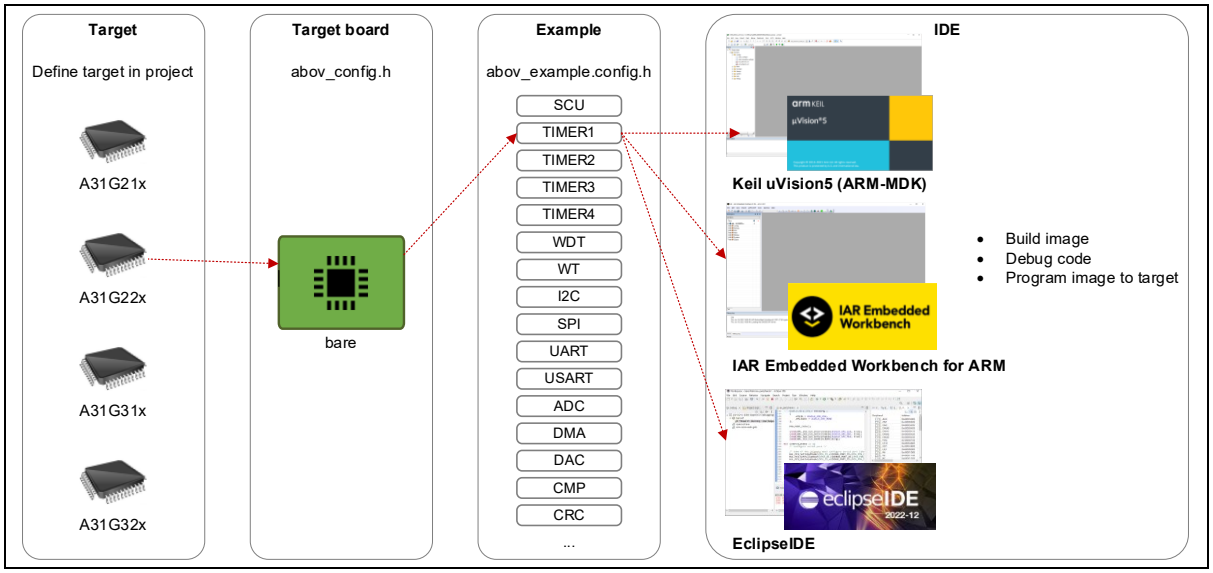
Bare board is a virtual H/W environment that is used to test the peripheral's operation without external H/W dependency. Therefore, the target board must be set to 'BARE' in AUDK32 to build the target example.

Exceptionally, users may be required to change the target example code regarding I/O pins, channels, external circuits, and other area which vary depending on the specific requirements for H/W platform.

For information on setting up AUDK32, refer to the document 'STD10-HB_AUDK32 Guide_Eng'.

Figure 1 shows a simple procedure for setting up a development environment where TIMER1 example running on the bare A31G22x board is configured and compiled.

Figure 1. Bare board configuration based on user input



1.1.2 Limitations

- If users select a peripheral example that is not supported on the target device, the users may see a warning message. To avoid displaying warning messages, refer to the user's manual to check which peripherals are supported on the target devices before selecting an example to test.
- Some peripherals could be scheduled to be supported soon.
- High level software components such as middleware or real-time OS will be available in the near future.

1.2 Script location

TeraTerm scripts and Minicom scripts are supported by AUDK32 basically.

You can find out the scripts under ~/Example/Macro/Minicom or TeraTerm in AUDK32 as below.

Figure 2. Script location

```
|-- Example
|   |-- Macro
|   |   |-- Minicom
|   |   |   `-- A31G21x
|   |   |       |-- ADC
|   |   |       |-- CFMC
|   |   |       |-- CRC
|   |   |       |-- DAC
|   |   |       |-- I2C
|   |   |       |-- LED
|   |   |       |-- PCU
|   |   |       |-- SCU
|   |   |       |-- SPI
|   |   |       |-- Timer1
|   |   |       |-- Timer2
|   |   |       |-- Timer3
|   |   |       |-- UART
|   |   |       |-- USART
|   |   |       |-- WDT
|   |   |       `-- WT
|   |   `-- TeraTerm
|   |           `-- A31G21x
|   |               |-- ADC
|   |               |-- CFMC
|   |               |-- CRC
|   |               |-- DAC
|   |               |-- I2C
|   |               |-- LED
|   |               |-- PCU
|   |               |-- SCU
|   |               |-- SPI
|   |               |-- Timer1
|   |               |-- Timer2
|   |               |-- Timer3
|   |               |-- UART
|   |               |-- USART
|   |               |-- WDT
|   |               `-- WT
|
```

2. Environment identification

2.1 Main components for S/W development

Table 2. S/W development environment

Component	Description	Comment
SDK version	AUDK 1.0.3 or higher	
Development IDE	MDK-ARM Essential: 5.34.0.0	supported by default.
	IAR Embedded Workbench for ARM: 8.50	supported by default.
	Eclipse Embedded C/C++: 2023-09 (4.29.0)	supported by default.
Host operating system	Windows 64bit	
	Ubuntu 22.04.3 64bit	

2.2 Build flags

Table 3. Keil build flags for S/W development

Build flags	Description
Assembly flag	--li -g --apcs=interwork --pd "__MICROLIB SETA 1" --pd "__UVISION_VERSION SETA 534" --list ".\Listings*.lst" --xref -o "*.o" --depend "*.d"
Compilation flags	--c99 -c -D__MICROLIB --li -g -O3 --apcs=interwork -D__UVISION_VERSION="534" -DEXTRN_ABOV_EXAMPLE_CONFIG -o .\Objects*.o --omf_browse .\Objects*.crf --depend .\Objects*.d
Link flags	--library_type=microlib --strict --diag_suppress L6314W --summary_stderr --info summarysizes --map --xref --callgraph --symbols --info sizes --info totals --info unused --info veneers --list ".\Listings\bare.map" -o .\Objects\bare.axf

Table 4. Eclipse build flag for S/W development

Build flags	Description
Assembly flag	-mthumb -mlittle-endian -Og -fmessage-length=0 -fsigned-char -ffunction-sections -fdata-sections -g3 -x assembler-with-cpp -MMD -MP -MF"$(@:%.o=%.d)" -MT"$@" -c -o "$@" "$<"
Compilation flags	-mthumb -mlittle-endian -Og -fmessage-length=0 -fsigned-char -ffunction-sections -fdata-sections -g3 -DEXTRN_ABOV_EXAMPLE_CONFIG -std=gnu11 -MMD -MP -MF"$(@:%.o=%.d)" -MT"$@" -c -o "$@" "$<"
Link flags	-mthumb -mlittle-endian -Og -fmessage-length=0 -fsigned-char -ffunction-sections -fdata-sections -g3 -Xlinker --gc-sections -Wl,-Map,"bare.map" --specs=nano.specs --specs=nosys.specs -Wl,--no-warn-rwx-segment -o "bare.elf" $(OBJS) $(USER_OBJS) $(LIBS)

3. Examples on bare board

This chapter describes basic CLI commands of the examples provided in AUDK32.

3.1 User interface

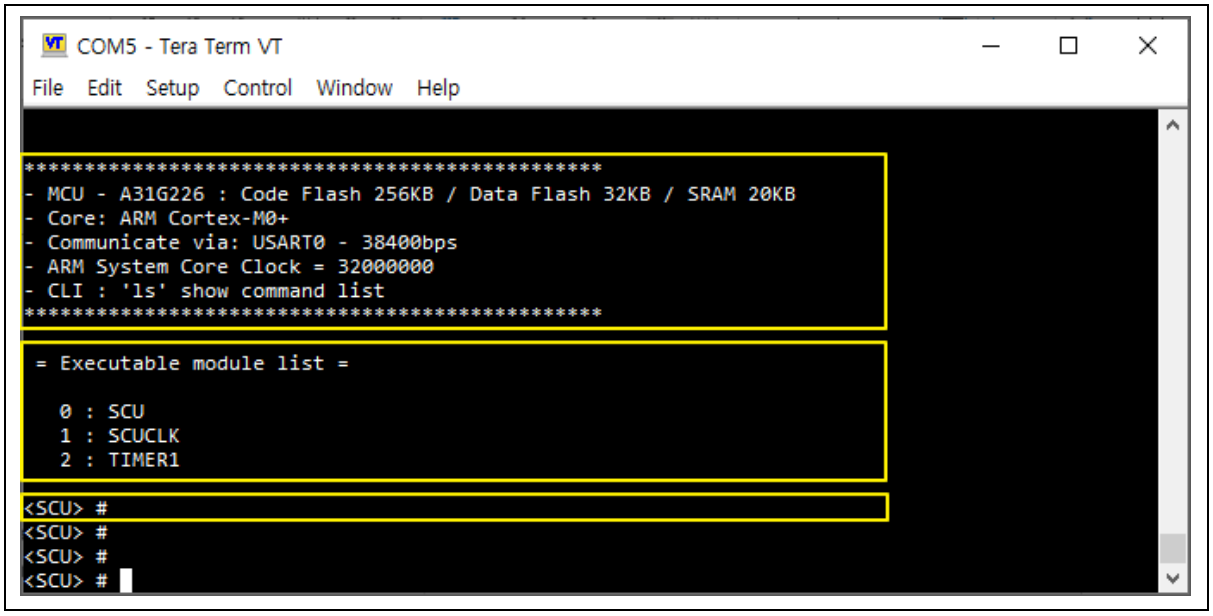
Since the operation and verification of the examples are implemented in CLI format by default, a terminal program such as TeraTerm or Minicom is required on host PC.

If users download the compiled image file onto the bare board after building a project, the users can see the system information and executable module list at the terminal, as shown in Figure 3.

- System information shows the information about ABOV device, serial debug channel and clock of the peripherals to test.
- Executable module list is a list of peripherals to operate. It includes SCU and SCUCLK by default required for an overall system and clock setup, and peripherals to test.

After showing the information and module list, a prompt blinks in the terminal to receive the command typed by user. The prompt displays its name as SCU by default and changes its name to a name of the peripheral selected from the executable module list.

Figure 3. Running example on bare target



3.2 Default commands

3.2.1 ls

The ‘ls’ lists top-level commands.

Figure 4. Result of ‘ls’ command

```
<SCU> # ls
 lm     : show module list
 cm     : change to specific module
    usage : cm [module] (one of module list by lm)
 help   : show specific module command list
<SCU> #
```

3.2.2 lm

The ‘lm’ lists modules to test.

Figure 5. Result of ‘lm’ command

```
<SCU> # lm
 = Executable module list =

   0 : SCU
   1 : SCUCLK
   2 : TIMER1

<SCU> #
```

3.2.3 cm

The ‘cm’ switches to the module to test.

SCU is selected by default. The target module can be changed from SCU to TIMER1 by using the cm command as shown in Figure 6.

Figure 6. Result of ‘cm’ command

```
<SCU> # cm
 lm     : show module list
 cm     : change to specific module
    usage : cm [module] (one of module list by lm)
 help   : show specific module command list
<SCU> #
```

3.2.4 help

The ‘help’ lists the test commands to operate and verify the target peripheral.

Figure 7. Result of ‘help’ command

```
<TIMER1> # help
h     : Usage
init : type 'h' to find out more information
uninit: type 'h' to find out more information
clk  : type 'h' to find out more information
config: type 'h' to find out more information
adctrgcfg: type 'h' to find out more information
start: type 'h' to find out more information
stop : type 'h' to find out more information
pause: type 'h' to find out more information
log  : type 'h' to find out more information
<TIMER1> #
```

3.2.5 h

The ‘h’ lists test commands and their options to operate and verify the target peripheral.

Figure 8. Result of ‘h’ command

```
<TIMER1> # h
=== Total number of Timer1 is 7. ===

Usage : 'h opt' display specific parameters
  init          [0 ~ 6]
  uninit        [0 ~ 6]
  clk           [0 ~ 6] [src] [opt] [prescale]
  config        [0 ~ 6] [ops] [mode] [opt] [pol] [data-a] [data-b] [-io]
  adctrgcfg     [0 ~ 6] [en] [mode:opt]
  start         [0 ~ 6]
  stop          [0 ~ 6]
  pause         [0 ~ 6] on / off
  log           on / off

<TIMER1> #
```

3.2.6 h opt

The ‘h opt’ outputs the detail of h command.

Figure 9. Result of ‘h opt’ command

```
<TIMER1> # h opt
=== Total number of Timer1 is 7. ===

Usage :
  init          [0 ~ 6]
  uninit        [0 ~ 6]
  clk           [0 ~ 6] [src] [opt] [prescale]
                  src  : p (pclk) / m (mccr) / e (ext)
                  opt
                  if select pclk : [div]
                    div  : 0, 2, 4, 16, 64
                  if select mccr : [mccr] [div]
                    mccr : l (lsi) / s (lse) / m (mclk) / h (hsi) / e (hse) / p (pll)
                    div  : 0 ~ 255
                  if select ext : [edge]
                    edge : f (falling) / r (rising)
                  prescale : 0 ~ N
  config        [0 ~ 6] [ops] [mode] [opt] [pol] [data-a] [data-b] [-io]
                  ops    : p (polling) / i (interrupt) / n (nmi)
                  mode   : p (periodic) / c (capture) / o (oneshot) / m (pwm)
                  opt
                  if select capture : [edge]
                    edge : r (rising) / f (falling) / b (both)
                  pol    : h (high) / l (low)
                  data-a : 0 ~ N
                  data-b : 0 ~ N
                  -io : i (input) / o (output) - in/output port
  adctrgcfg     [0 ~ 6] [en] [mode:opt]
                  en     : e (enable) / d (disable)
                  mode   : n (normal) / p (trigger point) / b (both)
                  opt
                  if p or b is selected : [val]
                    val  : 0 ~ N (16bit decimal) - trigger point counter value
  start         [0 ~ 6]
  stop          [0 ~ 6]
  pause         [0 ~ 6] on / off
  log           on / off

<TIMER1> #
```

Depending on the design characteristics of peripheral on a chipset, the detailed options here such as the number of input parameters of the command and the range of values may vary and there could be additional commands. Therefore, in order to use the command options correctly, users need to refer the peripheral specifications in the user’s manual of the target device.

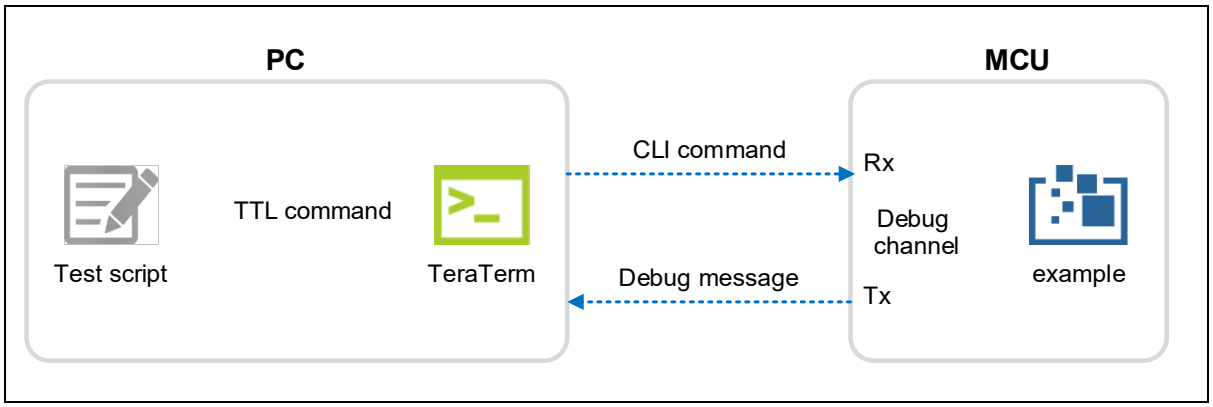
3.3 Peripheral test with terminal script

As described in introduction, Teraterm and minicom scripts are supported by default.
Therefore, it is recommended that Teraterm in Windows PC and Minicom in Linux PC are recommended.
In this chapter, testing peripheral with script is described in Window host PC only because there is no different engineering stuff except for macro syntax of user terminal program.

The examples in AUDK32 provide a set of CLI commands that allows users to test a peripheral by one of the two methods described below:

- The users type a CLI command on any serial host program directly.
  - The users have to consider about the composition of the commands provided by the example for a test scenario.
  - It is proper for a custom test scenario. However, in the case of a standardized test scenario, it is very low-productive and tedious.
- The users execute TeraTerm script file with the TeraTerm program.
  - The TeraTerm macros consist of a group of CLI commands to build up a standardized test scenario predefined and verified in advance as depicted in Figure 10.
  - The TeraTerm script file written in TeraTerm macro syntax and consist of a group of CLI commands on top of its native facility to build up a standardized test scenario defined and verified in advance as depicted in Figure 10.

Figure 10. AUDK32 example demonstration using TeraTerm script



3.3.1 Test script

The TeraTerm script files are implemented for each Sub-Family and can be found under the directory '~/Example/Macro/TeraTerm'.

Figure 11 shows the location of TeraTerm script files for each Sub-Family.

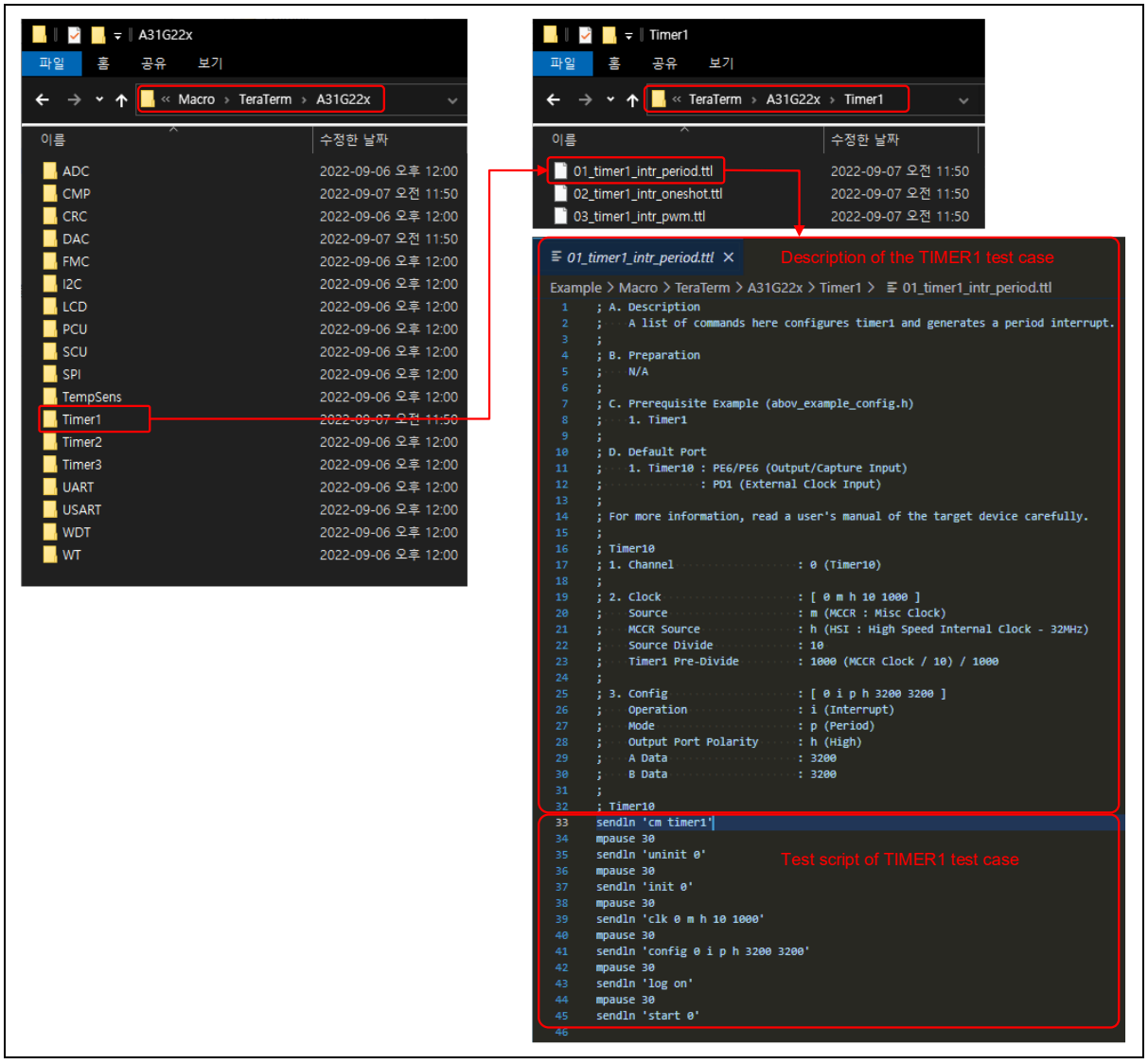
Figure 11. TeraTerm script file of peripherals

Figure 12 shows the TIMER1 test script file '01_timer1_intr_period.ttl' located at '~/Example/Macro/TeraTerm/A31G22x/Timer1', which is used to validate or test the periodic feature of TIMER 1 in the A31G22x Sub-Family."

The '01_timer1_intr_period.ttl' file includes the following information:

- The summary about TIMER1 test case and the option detail of CLI command to validate is described by comments.
- The script code for the TIMER1 peripheral test case is provided in a form of a combination of TeraTerm-based TTL commands and AUDK32's CLI commands just following the comments.

Figure 12. Example of test script

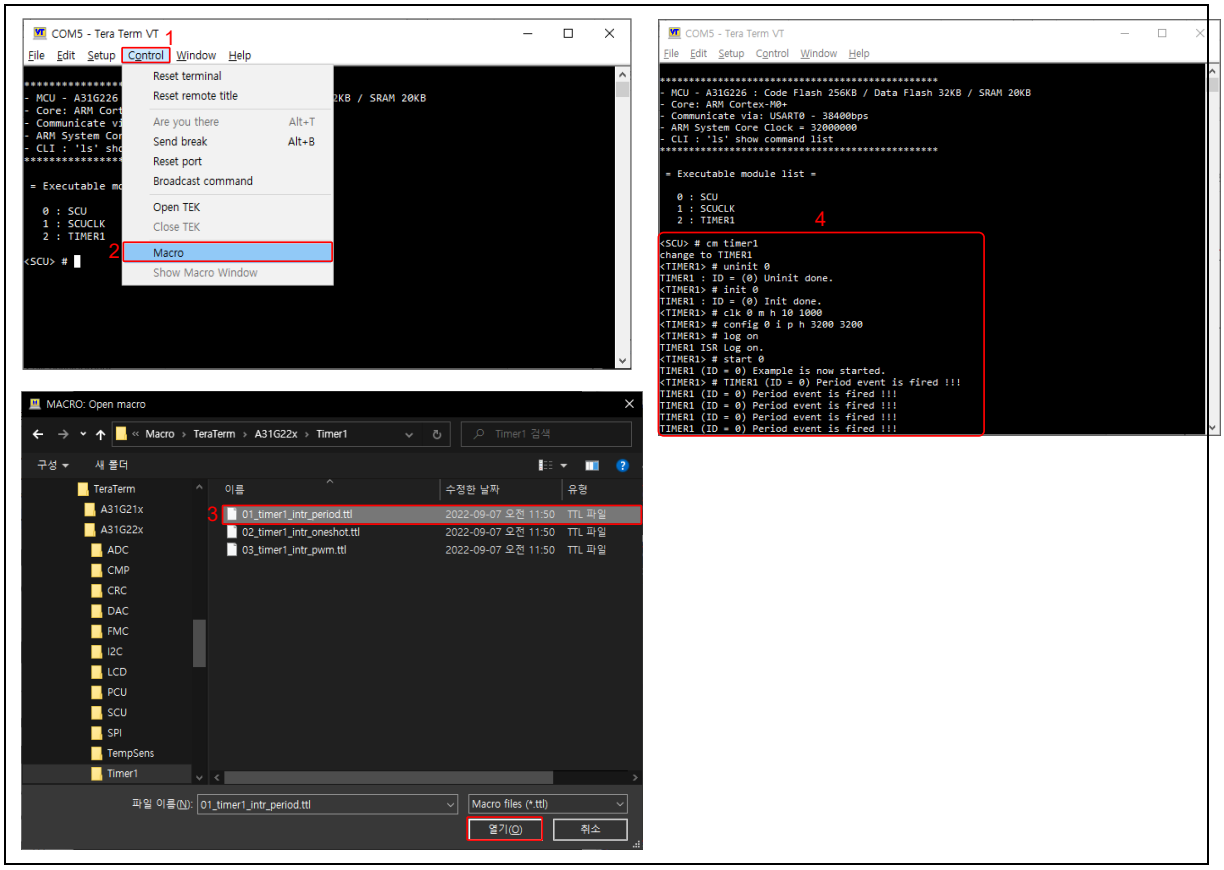


3.3.2 How to use test script

Figure 13 shows the procedure to test TIMER1 on the bare board mounting A31G22x which is the target device, using a test script:

1. Click the Control menu on the TeraTerm menu bar.
2. Click the 'Macro' from the Control dropdown menu.
3. Select the test script file '01_timer1_intr_period.ttl' at ~/Example/Macro/TeraTerm/A31G22x/Timer1/' in 'MACRO: Open macro' file browser.
4. Check the operation and results of the test script on the TeraTerm terminal.

Figure 13. Selection of macro file



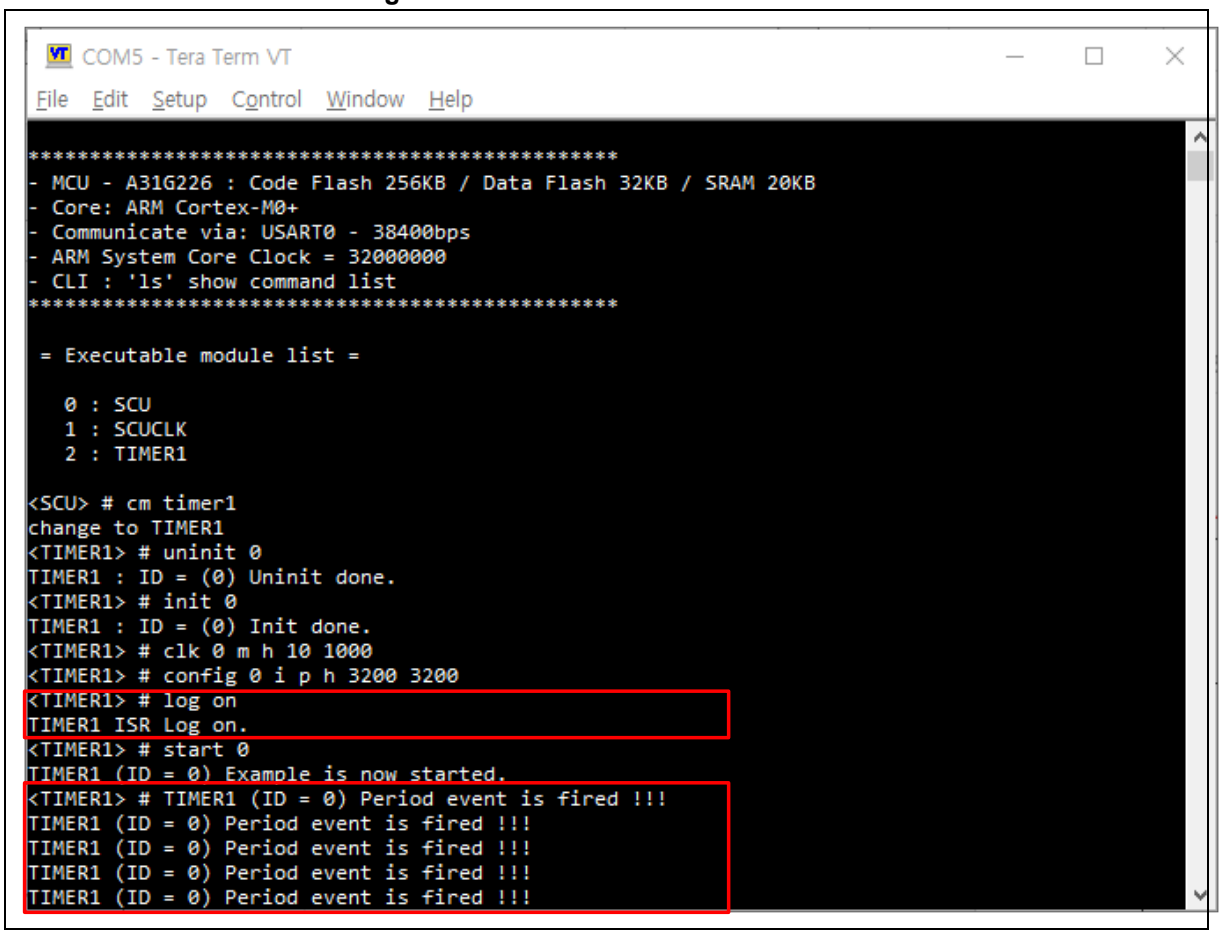
3.3.3 Checking the result detail of test script

Figure 14 displays the results of performing the test script '01_timer1_intr_period.ttl' shown in Figure 12.

Each CLI command is transmitted every 30ms and the result of it appears in TeraTerm program.

For more information on the '01_timer1_intr_period.ttl' script, refer to section 4.1.1.

Figure 14. Result of macro in TeraTerm



3.3.4 TeraTerm TTL command

For information on TTL commands of TeraTerm and related references, visit the URL below.

- https://ttssh2.osdn.jp/manual/4/en/macro/command/index.html

4. Test script detail

This chapter describes one of test scripts '01_timer1_intr_period.ttl' provided in AUDK32 for your understanding using TeraTerm only because there is no different engineering stuff except for macro syntax of user terminal program.
For minicom macro syntax, you can refer to https://linux.die.net/man/1/runscript.

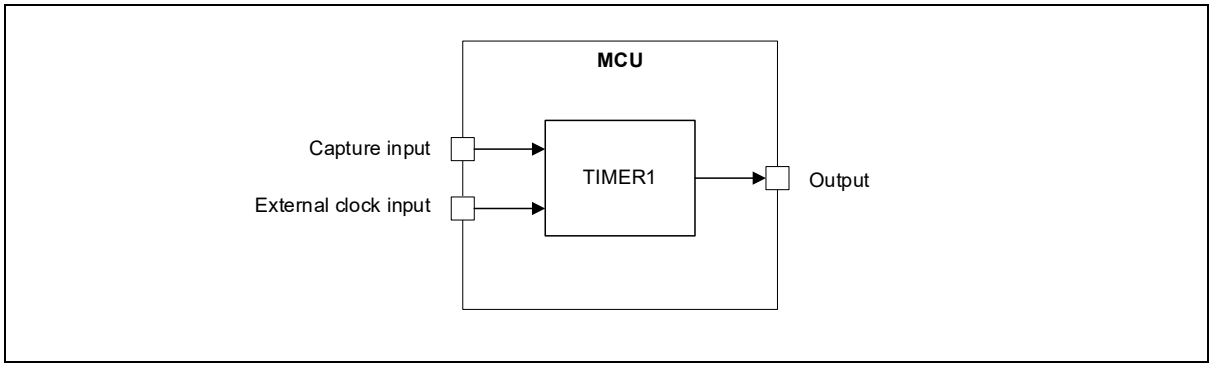
4.1 TIMER1

4.1.1 01_timer1_intr_period.ttl

This script configures timer1 for periodic and interrupt mode and starts TIMER1 up.

Figure 15 shows H/W configuration for this example.

Figure 15. H/W configuration block diagram of 01_timer1_intr_period.ttl




```
; A. Description
;    A list of commands here configures timer1 and generates a period interrupt.
;
; B. Preparation
;    N/A
;
; C. Prerequisite Example (abov_example_config.h)
;    1. Timer1
;
; D. Default Port
;    1. Timer10 : PE6/PE6 (Output/Capture Input)
;               : PD1 (External Clock Input)
;
; For more information, read a user's manual of the target device carefully.
;
; Timer10
; 1. Channel                    : 0 (Timer10)
;
; 2. Clock                      : [ 0 m h 10 1000 ]
;    Source                     : m (MCCR : Misc Clock)
;    MCCR Source                : h (HSI : High Speed Internal Clock - 32MHz)
```

```
22    ;     Source Divide              : 10
23    ;     Timer1 Pre-Divide          : 1000 (MCCR Clock / 10) / 1000
24    ;
25    ; 3. Config                     : [ 0 i p h 3200 3200 ]
26    ;     Operation                  : i (Interrupt)
27    ;     Mode                       : p (Period)
28    ;     Output Port Polarity       : h (High)
29    ;     A Data                     : 3200
30    ;     B Data                     : 3200
31    ;
32    ; Timer10
33    sendln 'cm timer1'
34    mpause 30
35    sendln 'uninit 0'
36    mpause 30
37    sendln 'init 0'
38    mpause 30
39    sendln 'clk 0 m h 10 1000'
40    mpause 30
41    sendln 'config 0 i p h 3200 3200'
42    mpause 30
43    sendln 'log on'
44    mpause 30
45    sendln 'start 0'
```

The TTL command ‘sendln’ at each line is performed every 30 ms (as shown at the lines 34, 36, 38, 40, 42, 44) by the TeraTerm program, and it sends out the CLI command to a debug channel of the target device.

The example TIMER1 running on target MCU gets the CLI command from TeraTerm program and does perform what it is expected by the CLI command.
When the operation is done, the example TIMER1 sends out the results of each CLI command to TeraTerm program via the debug channel.

Line 33: Switches the test module to TIMER1.

Line 35: De-initializes TIMER1 channel 0.

Line 37: Initializes TIMER1 channel 0.

Line 39: Sets up an operating clock of TIMER1 channel 0.

- m: Sets a clock source of TIMER1 channel 0 to SCU_MCCR clock.
- h: Sets a clock source of SCU_MCCR to HSI.
- 10: Divides HSI by 10. (HSI / 10 = 32 MHz / 10 = 3.2 MHz)
- 1000: Divides the input clock by pre-scaler, 1000. (3.2 MHz / 1000 = 3.2 kHz)

Line 41: Sets up operation mode of TIMER1 channel 0.

- i: Sets to interrupt mode.

- p: Sets to periodic mode.
- h: Sets the start polarity to high.
- 3200: Sets the match value of A count to 3200. (3.2 kHz / 3200 = 1 Hz)
- 3200: Sets the match value of B count to 3200. (3.2 kHz / 3200 = 1 Hz)

Line 43: Turns on log.

Line 45: Starts TIMER1 channel 0 operation.

Revision history

Revision	Date	Notes
1.00	Sep. 30, 2022	Initial release.
1.01	May. 09, 2023	• Fix typo. • Update asm, compile and linker options. • Update figures. • Correct mpause macro value.
1.10	Nov. 09, 2023	• Fix typo and incorrect expression. • Add Linux environment and the corresponding requirements. • Add Eclipse IDE and the corresponding requirements.

Korea

Regional Office, Seoul
R&D, Marketing & Sales
8th Fl., 330, Yeongdong-daero,
Gangnam-gu, Seoul,
06177, Korea

Tel: +82-2-2193-2200
Fax: +82-2-508-6903
www.abovsemi.com

HQ, Ochang
R&D, QA, and Test Center
93, Gangni 1-gil, Ochang-eup,
Cheongwon-gun,
Chungcheongbuk-do,28126, Korea

Tel: +82-43-219-5200
Fax: +82-43-217-3534
www.abovsemi.com

Domestic Sales Manager
Tel: +82-2-2193-2206
Fax: +82-2-508-6903
Email: sales_kr@abov.co.kr

Global Sales Manager
Tel: +82-2-2193-2281
Fax: +82-2-508-6903
Email: sales_gl@abov.co.kr

China Sales Manager
Tel: +86-755-8287-2205
Fax: +86-755-8287-2204
Email: sales_cn@abov.co.kr

ABOV Disclaimer

IMPORTANT NOTICE – PLEASE READ CAREFULLY

ABOV Semiconductor ("ABOV") reserves the right to make changes, corrections, enhancements, modifications, and improvements to ABOV products and/or to this document at any time without notice. ABOV does not give warranties as to the accuracy or completeness of the information included herein. Purchasers should obtain the latest relevant information of ABOV products before placing orders. Purchasers are entirely responsible for the choice, selection, and use of ABOV products and ABOV assumes no liability for application assistance or the design of purchasers' products. No license, express or implied, to any intellectual property rights is granted by ABOV herein. ABOV disclaims all express and implied warranties and shall not be responsible or liable for any injuries or damages related to use of ABOV products in such unauthorized applications. ABOV and the ABOV logo are trademarks of ABOV. All other product or service names are the property of their respective owners. Information in this document supersedes and replaces the information previously supplied in any former versions of this document.

© 2023 ABOV Semiconductor – All rights reserved